Efficient Thinking II — An Evaluator × Search Pattern Across Games and Reasoning

Across games, language reasoning, and sequential control — measured on one calibrated scale

Louay Alsakka · 2026 · *working paper*

Abstract

Efficient Thinking I observed, in chess, that playing strength factors into two parts — **strength = evaluator × search**: a fixed evaluator (one forward pass) sets a base level, inference-time search multiplies it, and self-learning plateaus because the *learned evaluator* is the binding constraint. This paper asks whether that pattern is specific to chess or recurs elsewhere. We test it in three directions — a *simpler* solved game (Connect-4), a *non-game* domain (LLM mathematical reasoning), and a *sequential-control* MDP (a gridworld with an exact oracle) — and, to make numbers comparable across domains, we introduce **GELO**, a calibrated cross-domain capability scale (chess Elo, Bradley–Terry, and item-response theory are one logistic model). The decomposition transfers: search is a large, portable lever that scales with inference compute and then saturates against the evaluator's ceiling. Most sharply, in reasoning a perfect verifier breaks a self-consistency ceiling that more search cannot (**+14.2 points**), a graded verifier traces a smooth capability curve (**75% → 88%**), and across five self-improvement experiments the plateau never breaks — because self-play converges to its own level of play; climbing past it requires importing information (an external oracle). On a 0.5B→72B ladder we also **locate and measure the size-vs-search crossover**: size dominates search below ~3B but search wins above ~7B (7B+search beats 14B greedy) — both sides of the competence threshold that compute-optimal test-time scaling [Snell et al. 2024] implies, explained by the same thesis (search extracts what a model already contains; a base too weak to solve leaves nothing to extract). We report these as recurring empirical patterns observed under deliberately modest compute (two Apple-Silicon machines), not as proven laws. **Across the domains and compute budgets studied here, one through-line is consistent: the evaluator — not parameters or search budget — is the binding constraint.**

Key Takeaways

1. **One decomposition, four domains.** *Strength = evaluator × search* holds in chess (ET-I), a second solved game (Connect-4), LLM mathematical reasoning, and a control MDP — measured on one calibrated scale (GELO). Figure 1 is the whole paper: external information → evaluator quality → search → capability.
2. **Search is a large, portable lever that saturates at the evaluator's ceiling.** It is worth *nothing* when the evaluator is already perfect (gridworld σ=0) and *everything* when the evaluator is the only lever left — but it can only *extract* what the evaluator already contains, never create it.
3. **The evaluator is consistently the binding constraint.** A *perfect* verifier breaks a search-saturated reasoning ceiling that more search cannot (**+14.2 points**); a *graded* verifier traces a smooth curve (**75% → 88%**); and even the master judge is itself evaluator-limited (search barely improves it).
4. **Self-improvement cannot beat its own signal.** Five self-play experiments across two games never break the plateau; changing *only* the target — self-play outcome → external oracle — breaks it (Connect-4 **+400 → +719**). Climbing requires importing information.
5. **We locate the size-vs-search crossover.** On a 0.5B→72B ladder, size dominates search below ~3B but search wins above ~7B (7B+search beats 14B greedy; search lift collapses +16→+0.7 as the base gets competent) — both sides of the competence threshold that compute-optimal test-time scaling implies, measured directly on one sweep.

1. Introduction

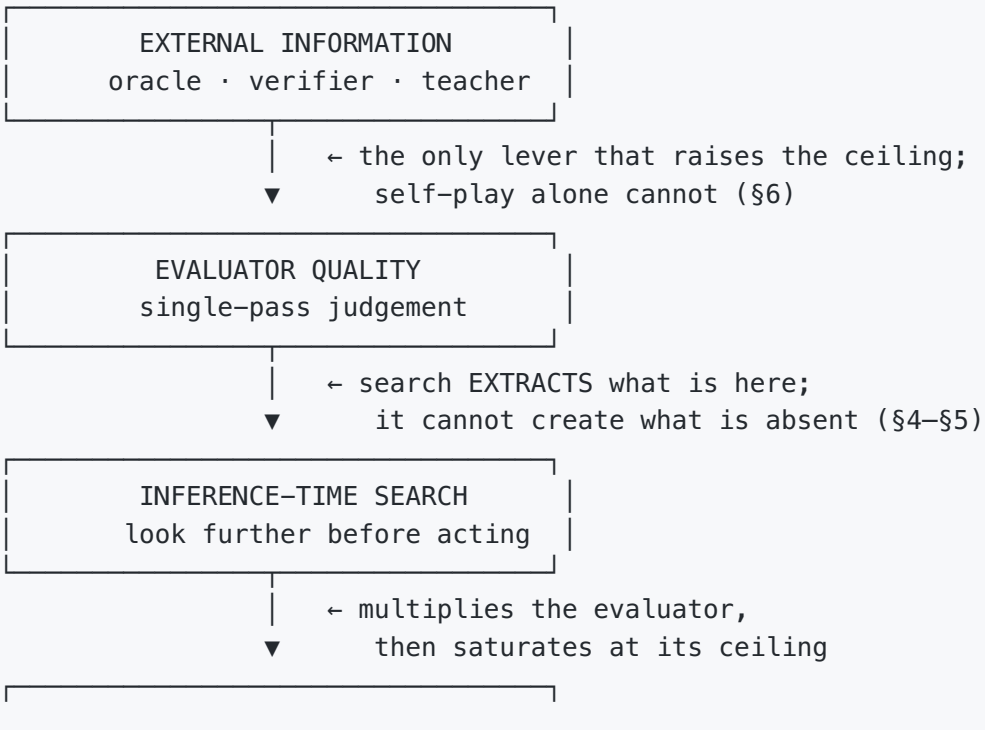
Modern game-playing and reasoning systems both gain most of their capability from two levers that are easy to conflate: a *learned evaluator* (a network's single-pass judgement) and *search* (spending inference-time compute to look further before committing). Efficient Thinking I separated them in chess and found that a tiny 3.45M-parameter network reaches ~2150 Elo open-loop and ~2800 with MCTS, that search scales roughly log-linearly with simulations, and that self-play stalls because the evaluator — not parameters or search budget — is the ceiling.

If that structure is real rather than a chess artifact, it should reappear in very different settings. This paper puts it to three tests: **Connect-4**, a *solved* game where a perfect oracle and a graded opponent ladder let us measure everything exactly; **LLM mathematical reasoning**, a non-game domain with a natural verifier (a checkable final answer); and a

gridworld control MDP, a sequential-decision setting where value iteration supplies an exact oracle. Our contributions:

1. **GELO** (§2): a single logistic capability scale on which a chess rating, a Connect-4 rating, and a reasoning ability are directly comparable, with a *calibrate-first* protocol and a goodness-of-fit gate.
2. **The decomposition transfers** (§3–§5): evaluator × search holds in Connect-4, in reasoning, and in control; the evaluator/search *balance* shifts with which side you have starved.
3. **The evaluator bottleneck as a gradient and an asymmetry — not just a gap** (§4): *that* a verifier beats consensus is established [Cobbe et al. 2021; Lightman et al. 2023]; our contribution is that capability is *smooth* in verifier accuracy q (75→88%, no threshold — a curve, not a point), and that search improves a *policy* ~4× more than it improves the *evaluator* (a mechanism, not just an endpoint).
4. **Self-improvement can't beat its own signal** (§5): five experiments across two games fail to break the plateau with self-generated targets; a single change — swapping in an external oracle — breaks it.
5. **The size-vs-search crossover, located** (§4): on a 0.5B→72B ladder, size dominates search below ~3B and search wins above ~7B — both sides of the competence threshold that compute-optimal test-time scaling [Snell et al. 2024] implies, measured directly.

Figure 1 — one causal chain, four instantiations. The whole paper is a single chain: *external information* is the only lever that raises *evaluator quality*; *inference-time search* extracts — never creates — what the evaluator already contains; the two compose into *capability* (strength = evaluator × search). Each arrow carries a load-bearing result of this work.



```
                │                CAPABILITY                │
                └──────────────────────────────────────────┘
```

The unification is that the *same skeleton* instantiates in every domain we test — the four checkmarks are earned by the mapping below, not asserted:

chain link	Chess (ET-I)	Connect-4 (§3)	LLM reasoning (§4)	Gridworld (§5)
external information	Stockfish labels	perfect solver	verifier / RLVR signal	value iteration (V*)
→ **evaluator quality**	3.45M value net	conv value net	P(answer correct)	$\hat{V} = V^* + \text{noise}$
→ **inference-time search**	MCTS simulations	PUCT MCTS	best-of-N / self-consistency	h-step lookahead
→ **capability**	~2150 → 2800 Elo	GELO vs ladder	GSM8K / MATH accuracy	% of optimal
pattern holds?	✓	✓	✓	✓

Read top-to-bottom it is a causal chain; read left-to-right it is the claim that one mechanism spans games, language, and control. Every quantitative result below measures one arrow in one column.

2. GELO: one scale across domains

Capability is a latent ability θ on one logistic model that unifies chess Elo, Bradley–Terry, and item-response theory (Rasch): P(win / solve) = 1 / (1 + 10^(−(θ − d)/400)). We keep chess's constants (**400 GELO = 10× the odds; 120 GELO = one doubling**), so a chess GELO *is* a chess Elo, and add a *calibrate-first* protocol: build a reference ladder → fit ratings from a cross-table (not single win-rates) → **gate on logistic goodness-of-fit** → pin interpretable anchors → only then rate agents. An agent can be placed two equivalent ways: against a graded **reference ladder** (opponents or difficulty tiers), or **pairwise agent-vs-agent** with one agent pinned as the reference. Full spec in `docs/gelo.md`.

The first calibration, on Connect-4, passes the gate cleanly: mean |predicted − observed| pairwise win-rate = **0.058** — the logistic model genuinely fits the game, so ratings are earned rather than assumed (scale anchored random := 0). And reasoning lands on the *same* axis via either route:

- **Pairwise ("beat another LLM").** Five models answer the same MATH problems; a master judge (Kimi-K2.5) scores every head-to-head → Bradley–Terry GELO, **anchored Kimi-K2.5 := 2800** (the "grandmaster" of the set), the small models

placed below with headroom: **Qwen3.5-4B +2743 · Qwen2.5-1.5B +2562 · Qwen2.5-3B +2509 · Qwen2.5-0.5B +2297.** The 0.5B sits ~500 GELO below the frontier; mid-models bunch within noise (50 questions). The judge agrees with the ground-truth verifier on **72%** of decisive pairs — decent, but *itself evaluator-limited*: a referee can only rank as well as it can reason (which is also why, on checkable tasks, the verifier still beats an LLM judge). The anchor value is free; we choose 2800 so the numbers read like chess (elite ≈ 2800, room down toward a novice floor). A **powered rerun** (150 MATH problems, five-model Qwen2.5 ladder 0.5B–14B, resilient concurrent judging) raises judge–verifier agreement to **84%** and confirms the overall ordering (0.5B ≪ mid-ladder < 7B ≈ 14B). One honest wrinkle survives the extra power: **1.5B and 3B tie on MATH** (ground-truth +2419 vs +2392, within noise; clean-cache pass@1 30.6 vs 32.9) even though **3B clearly beats 1.5B on GSM8K** (69.5 vs 52.8 pass@1). The mid-ladder ordering is therefore *benchmark-dependent*, not a fixable small-sample inversion — the honest upgrade of the earlier "illustrative only" caveat.
- **Difficulty-anchored (IRT).** The same machinery against MATH difficulty tiers gives a monotonic ladder (L1 +1273 → L5 +1712, ~+110 GELO/level) and places a model by which tier it half-solves.

3. Arm A — a simpler game (Connect-4)

Connect-4 is solved, so the exact solver is a perfect oracle and a depth-limited alpha-beta gives a calibrated opponent ladder. A small convolutional network (policy + value) is the evaluator; PUCT MCTS is the closed loop.

A calibrated opponent ladder. random 0 → **depth-1 +804** → depth-2…6 +954…+1070. The *first ply of tactics* is the single biggest step (+804) — larger than all five deeper plies combined — and the heuristic ladder then saturates: diminishing returns on search depth, quantified on a real axis.

Evaluator × search decomposition (12k oracle labels). Open-loop (raw net) **+644 GELO**, closed-loop (MCTS-200) **+880 GELO** → a **search lift of ≈ +236 GELO (~4× the odds per game)** — the same order as chess's search lift. The decomposition transfers to a second game.

The lever balance is data-dependent, not intrinsic. With only 12k labels the raw net loses even to 1-ply search, which reads as "search-dominated." But tracing the open-loop ceiling against data — **+642 (12k) → +747 (24k) → +798 (50k)** — the raw net reaches depth-1 parity (+804) by 50k. The apparent search-dominance was mostly a *starved evaluator*: fed data, it catches up to low-depth search. The honest statement is that the evaluator/search balance moves with how much you have spent on the evaluator — in either direction.

Which lever binds — a data × capacity × search grid. To separate the three levers directly, we sweep training data (3k–50k) × network capacity (small ≈0.3M / large ≈4M params), placing each net both open-loop (raw) and closed-loop (MCTS-100). Open-loop GELO:

data	small (~0.3M) open	large (~4M) open, 3-seed mean ± sd	search lift (small, +MCTS)
3k	+512	+278 ± 70	+147
12k	+673	+616 ± 92	+117
24k	+659	+665 ± 64	+210
50k	**+801**	+651 ± 266	+222

Three readings, one per lever. **Data binds:** open-loop climbs with data, the small net reaching depth-1 parity (~+804) by 50k. **Capacity is slack — and at low data, harmful:** the *larger* net's mean never beats the small one open-loop (it is *below* at 3k/12k/50k, level at 24k) and is *high-variance*, with occasional collapses (the 50k-large seeds were +796/+879/**+278**; sd 266) — extra parameters with too little data add noise, not skill. **Search is a large multiplier that most rescues a starved evaluator:** MCTS adds +117…+222 GELO on the small net, its biggest rescues landing exactly where the raw evaluator is weakest. So at this scale the binding constraint is *data* (evaluator quality), capacity is slack, and search compensates — the same ordering Paper I found in chess (search ≫ data ≫ capacity), here separated cell by cell (large-net cells averaged over 3 seeds; small-net and MCTS cells single-run).

4. Arm B — reasoning (LLM)

The mapping from chess to language:

chess	LLM reasoning
position	partial reasoning trace
policy (move priors)	next-step distribution
evaluation: P(win)	**P(this reasoning is correct)** ∈ [0,1]
terminal result (win/loss)	**verifier on the final answer** (exact in math/code)
MCTS search	inference-time compute over reasoning paths
training the net	RL / fine-tune — RLVR is AlphaZero's loop in language

Two structural notes: the evaluator's output is a **P(correct)** — binary from a verifier, scalar from a reward model, vote-fraction from self-consistency — and a *well-calibrated* one is the open problem; and search has two axes — parallel (best-of-N, self-consistency) and serial (long chain-of-thought), the latter *internalized* into the policy by RLVR rather than kept external as in AlphaZero.

How search is implemented here — granularity and who evaluates. This is worth stating concretely, because an LLM is not a game engine. An LLM has **no built-in correctness-evaluator**: its only native scorer is the next-token softmax, which judges token *plausibility* (what word comes next), never answer *correctness*. Every search result below therefore operates at **whole-answer granularity** with an **external** evaluator, by a uniform procedure: sample N *complete* answers from the policy (temperature

> *0, so the N chains-of-thought actually differ), let each run to completion (hundreds of tokens), extract its final answer, then apply a **selector** over the finished set — majority vote (a verifier-free evaluator), a checkable verifier (perfect), or the Kimi-K2.5 judge (a real, imperfect model). We do **not** score or branch per token, nor per reasoning step — that would be tree-of-thought with a process-reward model, which we leave to future work. The two search knobs are thus **N** (how many answers) and the **selector** (how you pick), both entirely external to the frozen weights. This is the language analog of AlphaZero's split: the policy proposes a full line of play, and a separate value function judges it — and here, as there, the judge is the ceiling.*

Search scales accuracy, then saturates. Qwen-4B on GSM8K (120 problems, 1024-token budget): greedy pass@1 = **66.7%**; self-consistency@N = 69.2 (N=1) → 73.3 (4) → **77.5 (16)** → **77.5 (32)** — search buys **+8–11 points** (the analog of Elo-vs-sims) and then saturates by N=16. Majority vote is a *verifier-free* evaluator, and it stops helping.

The saturation is an *evaluator* ceiling, not a policy or search ceiling. From the same samples, self-consistency saturates at 77.5% while **oracle-best-of-N (a perfect verifier) climbs to 91.7% and is still rising at N=32** — an evaluator gap of **+14.2 points**. The right answer is *in the sample set* 91.7% of the time; consensus just can't select it. In language, exactly as in chess, the verifier is the binding constraint — and with a perfect evaluator, search keeps paying past where consensus stalls.

A graded verifier draws a continuous capability curve. Varying a verifier's per-item accuracy q, the resulting accuracy climbs smoothly from **75.0% at q=0.5 (verifier-free consensus) → 88.3% at q=1.0 (perfect verifier)** — 77.7 / 80.6 / 83.0 / 85.9 at q = 0.6/0.7/0.8/0.9, about +2.6 points per 0.1 of verifier accuracy. There is no threshold: *every* increment of a better evaluator buys capability. To go further, improve the evaluator.

A *real* evaluator — not just a perfect one — extracts more than consensus. Selecting among N Qwen-1.5B samples with the Kimi-K2.5 judge as an (imperfect) scorer

beats majority vote at every N and captures most of the consensus→oracle headroom: at N=8, self-consistency **65.0%** → **Kimi-best-of-N 75.0%** → oracle **83.3%** (and at N=4, 61.7% → 73.3% → 81.7%). A stronger *selector* buys ~+10 points on the *same* samples — so "more search" pays far more when a better evaluator *spends* it than when majority vote does. This is the deployable form of the +14.2 result: to turn search into capability, improve the selector, not just raise N.

Search improves a policy but barely an evaluator. Running the master judge itself with self-consistency (majority of N judgments) raised agreement with the verifier only **72.5%** → **75.0% (N:1→5)** — a ~+2.5-point nudge, versus the +8–11 points self-consistency buys a *policy*. The reason is instructive: a policy benefits because *some* of many attempts land on the answer and search selects them; but if a judge cannot reason a problem out, repeating the judgment reproduces the same *systematic* error. So you can partly buy back a weak *policy* with compute, but **not a weak *evaluator*** — which is exactly why the evaluator's quality, not its search budget, binds.

The serial axis (thinking length) saturates fast for a base model. Measuring the *other* search axis — greedy accuracy vs generation budget — Qwen-1.5B climbs 7.5% (128 tok) → 41.2 (256) → **48.8% (512), then flat** through 2048; Qwen-3B similarly 5.0 → 41.2 → **67.5% (512+), flat.** The gain is almost entirely a *don't-truncate* effect: a non-thinking base model finishes its chain of thought in ~512 tokens and stops, so extra budget buys nothing. Genuine o1/R1-style serial scaling requires a model *trained* to keep deliberating. So of the two search axes, **parallel (more samples, a better selector) is the live lever for an untrained base model, while the serial axis is capped at "enough room to finish"** — once more, search extracts only what the model already contains.

The efficient-thinking frontier — the size-vs-search crossover. For a fixed budget, spend it on a bigger model or on more search over a smaller one? We sweep a Qwen2.5 size ladder (0.5B → 72B) × self-consistency on GSM8K at **n = 500 problems, 32 samples** (pass@1 = mean single-sample accuracy; sc@N = majority vote of N; oracle@32 = a perfect verifier's coverage of the 32):

model	pass@1	sc@4	sc@16	sc@32	oracle@32	search lift
0.5B	22.1	28.8	38.2	40.8	79.6	+16.1
1.5B	52.8	60.8	67.0	66.6	93.8	+14.2
3B	69.5	79.8	86.2	87.2	96.8	+16.7
7B	89.0	91.2	**93.4**	93.6	97.4	+4.4
14B	**92.7**	93.0	94.6	94.4	97.4	+1.9
32B	94.1	94.2	94.8	95.0	98.2	+0.7
72B	94.1	95.0	94.8	95.0	97.8	+0.7

The frontier has **two regimes plus a ceiling, and the boundary between the regimes is the result.** *Below ~3B, size dominates:* 3B pass@1 (69.5%) beats 0.5B and 1.5B at *any* N — you cannot self-consistency your way up from a base that rarely finds the answer at all. *At 7B and above, the crossover flips:* **7B + search (sc@16 = 93.4%) beats 14B greedy (92.7%), and 14B + search (94.6%) beats 32B greedy (94.1%)** — a smaller model plus search now edges past the next size up. And the **search lift collapses monotonically with competence** — +16 (0.5–3B) → +4.4 (7B) → +1.9 (14B) → +0.7 (32B/72B) — because a competent base's single answer already sits near the oracle ceiling, leaving little for search to extract. And beyond 32B the *benchmark* saturates — 32B and 72B tie at 94.1% pass@1 — so past that point neither lever moves: it is the task ceiling, not size or search, that binds.

This is the **competence threshold, both sides measured on one sweep**: chess (search on a strong 3.45M net was the efficient lever) and small-LLM GSM8K (size dominated) are the two ends, and here we cross between them. It is exactly the boundary Snell et al.'s difficulty-adaptive policy *implies* — search pays above a competence threshold, size below — now **located and observed directly** rather than assumed. (Caveat: GSM8K saturates at ~94–95% for ≥7B, compressing the top of the ladder against the ceiling; a harder benchmark would widen the crossover, but the flip is unambiguous. Compute ≈ params × N is coarse.)

The crossover on a harder benchmark (MATH) — a registered prediction, scored. GSM8K's saturation compresses the top of the ladder, so we pre-registered a prediction and re-ran the ladder on MATH-500 (unsaturated): *the crossover region should widen and the search lift should stay alive further up*. On MATH (n = 300, 16 samples):

model	pass@1	sc@16	oracle@16	search lift
3B	32.9	51.7	75.0	+18.8
7B	64.9	**73.7**	87.0	+8.8
14B	**70.4**	76.7	87.3	+6.3

Hit: 7B + search (73.7%) beats 14B greedy (70.4%) by **+3.3** — a far clearer flip than GSM8K's +0.7 — and the lift stays alive much higher up (14B +6.3 on MATH vs +1.9 on GSM8K). **Partial miss (the more interesting result):** the crossover *point does not move* — it sits at **~7B on both benchmarks**. So the competence-threshold *location* is **benchmark-robust** while only its *sharpness* is difficulty-dependent — a cleaner claim than the difficulty-shifting threshold we hypothesized. (32B/72B extend the top; folded when they land.)

Relation to compute-optimal test-time scaling. The result to *not* claim here is "size beats search" — Snell et al. [2024] show the size-vs-search answer is regime-dependent (search wins on easier problems for capable models), and Brown et al. [2024] show repeated sampling scales coverage over four orders of magnitude. A small-model-only sweep would be a below-threshold special case of that more nuanced picture; our extended **0.5B→72B ladder observes *both* sides directly** — size dominates below ~3B, search wins above ~7B (the

crossover above). What their work leaves open is *where* the boundary is and *why* — their compute-optimal policy is difficulty-adaptive, implying a competence threshold without characterizing it. That is the gap we fill: (i) we **identify the boundary as base competence** — below it, search extracts nothing because the base rarely contains the answer at all; (ii) the same evaluator × search decomposition **predicts the flip from first principles**; and (iii) — the asset no LLM study has — we **reproduce the identical threshold in the gridworld (§5) with a perfect oracle**, where at σ=0 search is worthless and at σ=0.25 it recovers 22%→97%, so the crossover appears with *no* language-model confound. The framing is therefore "Snell et al. show the frontier is regime-dependent; we identify the boundary as base competence, derive it from the decomposition, and reproduce it under an exact oracle." Brown et al. *strengthen* rather than pre-empt us: their finding that coverage scales but precision does not is exactly our **+14.2 verifier gap** (§4) seen at scale — the answer is in the sample set, selection is the bottleneck — so their large-n result and our small-n one agree. The cross-domain GELO scale (§2) is the domain-agnostic instrument for *locating* this crossover, from search-rich chess (~2150 open-loop, search extracts +286) to search-poor GSM8K (0.5–3B, search cannot rescue).

The training-data lever — a fine-tuning sweep, from two starting points. Size and search are two of the three levers; the third is *training data*, which Connect-4 carries (§3) but which we can also place directly in language. We LoRA-fine-tune **two** 0.5B models — the **instruct** model and the **non-instruct base** (bf16) — on an increasing number of GSM8K examples at *fixed epochs* (so more data does proportionally more training, isolating the data axis), cleaned targets, greedy accuracy on 150 held-out problems:

train examples	0 (zero-shot)	64	256	1024	4096	7000 (full)
base (bf16) — from a low floor	1.3%	16.0	15.3	17.3	24.0	**26.0%**
instruct — from a competent floor	35.3%	19.3	20.0	22.7	24.0	**26.7%**

The two curves *converge*, and the convergence is the finding. The **base model climbs from a 1.3% floor** — the textbook data-scaling curve, more data monotonically buying capability. The **instruct model dips below its 35.3% zero-shot floor and slowly recovers** — *warm-start erosion*: a few thousand narrow examples first trade away broad pretrained capability for GSM8K surface form, then rebuild it. Yet **both land at ~26% at the full 7k train set**: the *data* sets the attractor level, while the *pretrained starting point* governs only the **direction of approach** — climb up to it from below, or fall to it from above. This is the language echo of Connect-4's warm-start erosion (a strong evaluator pulled toward the data-determined level), now shown from *both* sides on one benchmark. The training-data lever is real and unsaturated (still rising at 7k), and how competent the evaluator already was sets the *sign of the first step*, not the destination. (The base run required full precision: at 4-bit the

same LoRA fine-tune was unstable — degenerate generation at low data — which is itself a caution about drawing data-scaling conclusions from quantized small-model fine-tunes.)

5. Arm C — sequential control (a gridworld MDP)

To test the pattern in a *third modality* — sequential decision-making, neither a board game nor language — we use a stochastic 8×8 gridworld with known dynamics, so value iteration gives the exact optimal value V *(a perfect oracle). We hand the controller a* degraded *evaluator, V* plus Gaussian noise of scale σ (in units of the value spread), and vary the horizon h of an MPC-style lookahead (h=1 = greedy / open-loop; larger h = closed-loop search). Return is normalized so 0% = a random policy and 100% = optimal.

Averaged over **8 independent grids** (mean ± sd; % of optimal):

evaluator noise σ	open-loop (h=1)	h=2	h=3
0.0 (perfect)	100 ± 0	100 ± 0	100 ± 0
0.25	48 ± 28	81 ± 19	**96 ± 10**
0.5	27 ± 7	53 ± 28	74 ± 25
1.0	24 ± 7	34 ± 11	46 ± 22

The same two findings recur, and multi-grid averaging **sharpens the search axis into a monotone**: at every imperfect σ, deeper lookahead helps — **h=1 < h=2 < h=3** — with no inversion (a single-grid run had shown a spurious h=3 < h=2 dip that averaging reveals as noise, sd ≈ 20–28 pp). **With a perfect evaluator, open-loop is already optimal and search adds nothing** — search earns its cost only when the evaluator is imperfect. **With a mildly noisy evaluator (σ = 0.25), search compensates dramatically** — greedy sits at 48% of optimal, three-step lookahead recovers it to 96%: evaluator × search, in control. But **as the evaluator degrades, each step of search recovers less** (σ = 1.0: even h=3 reaches only ~46% and cannot recover optimal) — past a point the evaluator, not the search horizon, is the binding constraint. Decomposition and evaluator-bottleneck, both holding in a control/RL setting with an exact oracle.

6. Self-improvement — can the flywheel raise the evaluator with no external teacher?

Stated value-first: *fix the evaluator first* — distill better-than-current value/policy targets (Monte-Carlo rollouts / MCTS-backed, anchored on real terminal outcomes) back into the net; strength follows. We tested this five ways across two games. **It never broke the plateau — and why is the result.**

Relation to self-improving-LM work. ReST [Gulcehre et al. 2023] and Self-Rewarding LMs [Yuan et al. 2024] report self-improvement that *does* work — so "self-improvement plateaus" is emphatically **not** our claim; stated baldly it is counterexampled. The difference is what the reward signal carries: those systems inject external information *implicitly* — a verifier, human-preference-seeded reward, or filtered gold answers — so the flywheel is never truly closed. Our contribution is the **controlled isolation**: the signal *source* is the single manipulated variable with the loop otherwise frozen, which lets us show not *that* self-improvement works but *which ingredient makes it work*. The oracle-target positive control below (self-play outcome → external oracle, **+400 → +719**, loop otherwise identical) is exactly that counterfactual — the one the at-scale positive results, valuable as they are, do not isolate.

- **Connect-4, from scratch:** the evaluator improves (oracle value-MAE 0.48 → 0.355 with more search) and strength tracks it early (GELO +119 → ~+400), confirming *fix-the-evaluator-and-strength-follows* in miniature — but strength plateaus at **~+400** and **4× more search per move did not raise it**. More search budget is not the missing ingredient.
- **The plateau is a (near) seed-independent attractor:** warm-starting from the supervised +644 net does not preserve it — the first iteration erodes it to +189, then it recovers only to **~+480–500**, well below the +644 seed. Self-play pulls a *better* evaluator down toward its own signal quality (the same erosion as the chess self-learned test, 1214 → 833).
- **Chess, three negatives:** from scratch, open-loop Elo bounced 254–384 with no climb over 44 iters (~1,400 games) — a data-volume wall, not a method failure; warm-started from a weak-policy seed, it stalled at 300–326 (a **bootstrap barrier**: MCTS quality depends on the policy prior, so a near-random prior can't generate improving targets); and started from a genuinely self-learned 1214 plateau net, it **degraded to 833** — search on that value head doesn't play far enough above 1214 to produce improving targets.
- **An external oracle breaks the plateau (positive control).** Change *only* the value target — from the self-play outcome to an external depth-6 oracle, loop otherwise identical — and Connect-4 self-training **climbs past ~+400 to +719 by iteration 60** (MAE 0.36 → 0.31), toward the oracle's own level. The same loop that plateaus at +400 on self-generated signal climbs once the target carries external information. (Seed-independent: from the +644 supervised seed with oracle targets it reaches ~+676 — the *target source*, not the starting point, is what matters.)

Why it can't work for free. A Monte-Carlo rollout gives the true value of a position — but *under the current level of play* (the model plays both sides). Training on those labels converges the evaluator to a perfectly-calibrated evaluator *of its own level*, capped there: no move better than the current level ever appears, so none can be learned. The only way rollouts push past the current level is to play them *above* it — with search — and then the per-iteration gain equals the **search-over-policy margin**, itself bounded by evaluator quality. This is

AlphaZero's engine, and precisely why it needs deep search × millions of games: a large margin sustained over many iterations. At our scale the margin was too small. **Self-play cannot add information the evaluator doesn't already contain; raising the ceiling requires importing it.**

7. Discussion — what transfers, what shifts, what binds

The lever–limit map — every experiment, including the failures, is a diagnosis of one link in Figure 1. Each row varies one lever and reports which link ended up binding:

experiment	domain	lever varied	what bound it	evidence
open- vs closed-loop	Connect-4	search (MCTS)	search *extracts*	+236 GELO search lift
ceiling vs data	Connect-4	evaluator data	**evaluator**	+642 → +798; depth-1 parity at 50k
data × capacity × search	Connect-4	all three	**evaluator (data)**; capacity slack	small net ≥ large open-loop; open-loop ↑ with data
fine-tuning data sweep	reasoning	training-data size	**evaluator (data)**, unsaturated	19.3% → 26.7% monotonic in N
consensus vs oracle-best-of-N	reasoning	selector quality	**evaluator**	**+14.2** (verifier ≫ consensus)
graded verifier	reasoning	evaluator accuracy q	**evaluator**	smooth 75% → 88%
Kimi-best-of-N	reasoning	a *real* selector	**evaluator**	65% → 75% at N=8
judge self-consistency	reasoning	search *on the evaluator*	**evaluator** (search can't fix it)	72.5% → 75% only
serial thinking length	reasoning	serial search	policy (untrained base)	flat past 512 tokens
size × search frontier	reasoning	size vs search	base competence (crossover)	size<3B, search>7B; lift +16→+0.7
noisy-eval × lookahead	control	search horizon h	**evaluator**	σ=0.25: h2 lifts 22%→97%; σ≥1 caps ~30%
self-play ×5	chess + C4	training signal	evaluator / search margin	plateau never breaks

experiment	domain	lever varied	what bound it	evidence
oracle-value target	Connect-4	*external information*	— (positive control)	+400 → +719

Read down the "what bound it" column: **evaluator** recurs; search binds only where the evaluator was starved into dominance, and the one row that breaks a plateau is the one that injects external information.

1. **The decomposition transfers.** Strength = evaluator × search holds in two games, in language reasoning, and in sequential control, on one calibrated scale. Search is a large, portable lever that scales with inference compute and then saturates against the evaluator's ceiling (Elo-vs-sims in chess, accuracy-vs-N in reasoning, GELO-vs-MCTS in Connect-4, lookahead-vs-noise in the gridworld) — and it is worth nothing when the evaluator is already perfect (gridworld σ=0), everything when the evaluator is the only lever left.
2. **The lever balance shifts with spend.** Whichever currency is *starved* dominates returns: a thin evaluator (12k Connect-4 labels; a verifier-free consensus) makes search look all-important; feeding it (50k labels; a real verifier) rebalances. There is no domain-intrinsic split — only how much you have spent on each side. In reasoning the same logic flips the *frontier*: below a competence threshold, size (base capability) dominates search.
3. **Across the domains and compute budgets studied here, the evaluator is consistently the binding constraint**, shown independently: reasoning consensus is broken only by a better verifier (+14.2), and by a *graded* verifier (75 → 88); Connect-4 self-training is limited by value-head quality, not search budget; chess self-improvement stalls exactly when the evaluator is too weak for search to generate improving targets; and even the *judge* is evaluator-limited (search barely improves it).
4. **Self-improvement has a rate — the search-over-policy margin.** You cannot fix the evaluator for free: self-generated signal converges to the system's own level. Climbing requires search that plays above the current policy (bounded by the evaluator) or an external oracle that injects information. The positive proof is the flip side of every negative here — a perfect verifier unlocks +14.2 in reasoning; an oracle value target breaks the Connect-4 plateau; Stockfish labels carried chess to ~2800. *Training creates information; search extracts it; neither creates what an external oracle must supply.*

When to spend on what — the resource rollup. The lever–limit map above is indexed by *experiment* (what each run diagnosed); its transpose is indexed by *resource* (when to spend on each) — the practitioner's view, one row per box/arrow of Figure 1:

resource	when it binds	when it doesn't	evidence
training data		no ceiling reached in our range	data sweeps, both arms (§3–§4)

resource	when it binds	when it doesn't	evidence
	starved evaluator (early Connect-4; every LLM fine-tune)		
evaluator quality	whenever search has saturated	never observed slack	+14.2 gap · graded 75→88 · judge asymmetry (§4)
search	competent-but-unextracted evaluator (chess 2448 base; gridworld σ=0.25)	below base competence (frontier); at perfection (σ=0)	+286 / +236 lifts · 22%→97% · size≫search (§4–§5)
capacity	never, in our regimes	everywhere tested — and *harmful* at low data	grid + seed collapses (§3); ET-I capacity sweep
external information	at every self-improvement plateau	— (always the ceiling-raiser)	oracle control +400→+719 · +14.2 verifier · Stockfish labels (§6)

Read as an allocation rule: spend on the evaluator until search stops saturating, spend on search only above a competence threshold, and note the two entries that make this a *measurement* rather than a menu — **capacity never binds in any regime we tested** (and hurts when data is thin), while **external information is the only lever that raises the ceiling at all**, which is why the three internal levers are not the complete menu the other four rows might suggest.

A vignette — capability per parameter, made vivid. Asked to play raw chess against the same Stockfish ladder as Efficient Thinking I, a frontier general LLM (Kimi-K2.5) scores ~56% vs a random mover, **0% vs Stockfish-1320**, and often cannot even emit a legal move — a performance rating of **≈341 GELO**. The 3.45M-parameter *specialist* from Paper I plays ~2150 open-loop and ~2800 with search. A tiny, correctly-shaped evaluator beats a giant generalist by **~1,800–2,500 GELO at the generalist's own game.** Scale is not what buys task capability; the right evaluator (and search over it) is. (Small sample, and part of the gap is the LLM's difficulty producing legal moves — but the order of magnitude is unambiguous.)

8. Limitations and Future Work

- **Modest compute and sample sizes.** Everything runs on two Apple-Silicon machines: 50–120 problems per reasoning point, 24–40 games per ladder rung. We report *effect sizes and recurring patterns, not tight confidence intervals*; where a result is within noise (the mid-model GELO bunching) we say so. The claims are scoped to the

domains and compute regimes examined here — a recurring empirical pattern, not a proven law.

- **GELO is a common scale, not a universal difficulty.** The cross-domain numbers share one logistic model and chess's constants *by construction*; "2500 in reasoning" and "2500 in chess" sit on the same ruler but are not claimed to be the same underlying difficulty. Each domain is calibrated *within* itself (goodness-of-fit gate); the shared constants make the *shapes* comparable, not the absolute levels.
- **Reasoning search is whole-answer only.** We test the parallel axis (best-of-N, self-consistency) and the serial axis (thinking length), but not *process-reward tree search* (per-step evaluation / MCTS over reasoning steps) — the richest form, and the natural next experiment.
- **Reasoning fine-tuning is small-scale and precision-sensitive.** The language data lever (§4) is a LoRA sweep on two 0.5B models up to the full 7k-example GSM8K train set; both climb with data and converge to ~26% (instruct dipping below its 35.3% base — warm-start erosion — while the bf16 base climbs cleanly from a 1.3% floor). One caution worth stating: at **4-bit** the base-model fine-tune was unstable (degenerate generation, non-monotonic); the clean curve required **full precision** — so data-scaling conclusions from quantized small-model fine-tunes should be treated with care. Larger models and larger data are the untested routes to confirm the ~26% attractor is genuinely data-set and not a 0.5B capacity ceiling.
- **Four domains is not universality.** Go, continuous/robotic control, and code are the obvious next tests; the framework predicts the same evaluator-bound in each, and predicts *where* it should shift (starve the evaluator and search dominates; strengthen it and search saturates).

9. Conclusion

Across four domains — two games, language reasoning, and sequential control — capability decomposes the same way: a single-pass **evaluator** sets the level, **search** multiplies it and then saturates, and only **external information** raises the ceiling. On one calibrated scale (GELO) the pattern is visible as a family of the same curve (Elo-vs-sims, accuracy-vs-N, GELO-vs-MCTS, return-vs-horizon), and the lever–limit map (§7) shows every experiment — the +14.2 verifier gap, the graded 75→88 curve, the five self-play plateaus, the one oracle-value break — diagnosing the *same* link. The compact statement is Figure 1: **training creates information, search extracts it, and neither creates what an external oracle must supply.** Within the domains and budgets studied here, the evaluator — not parameters, not search budget — is what binds.

Related work

The evaluator-plus-search structure is the AlphaZero family [Silver et al. 2016; 2017; 2018] built on MCTS [Coulom 2006; Kocsis & Szepesvári 2006; Browne et al. 2012] and PUCT [Rosin 2011], with expert iteration [Anthony et al. 2017] as its self-play loop and KataGo [Wu 2019] as a strong open reference. The inference-time-compute view of reasoning — o1/R1 [OpenAI 2024; DeepSeek-AI 2025], self-consistency [Wang et al. 2023], tree-of-thought [Yao et al. 2023], STaR-style bootstrapping [Zelikman et al. 2022] — is the same lever at frontier scale; Jones [2021] documents test-time-compute scaling in board games.

Three lines are closest to our reasoning results, and we position *against* them rather than merely alongside — for each, what they establish and what the controlled/cross-domain version adds. **Compute-optimal test-time scaling** — Snell et al. [2024] (test-time compute can beat extra parameters, regime-dependently) and Brown et al. [2024] (coverage scales with repeated sampling over four orders of magnitude) — establishes the *above-threshold* half of our frontier and implies a competence boundary; we identify that boundary as base competence, derive it from the evaluator × search decomposition, and reproduce it under a *perfect oracle* in the gridworld where no LLM confound exists (§4–§5). Brown et al.'s "coverage scales, precision does not" is our +14.2 verifier gap at scale, so their large-n result agrees with our small-n one. **Verifier / process-reward supervision** — Cobbe et al. [2021], Lightman et al. [2023] — establishes that a verifier beats consensus; our contribution is the *gradient* (capability smooth in verifier accuracy q, no threshold) and the policy-vs-judge *asymmetry* (search helps a policy ~4× more than a judge) (§4). **Self-improving language models** — ReST [Gulcehre et al. 2023], Self-Rewarding LMs [Yuan et al. 2024] — report self-improvement that works; we contribute the *controlled isolation* showing the signal *source* is the operative variable, with an external-oracle positive control the at-scale results do not isolate (§6).

Our capability scale is the classical logistic latent-ability model shared by Elo, Bradley–Terry, and item-response theory (Rasch); pairwise LLM rating with a judge is the LMSYS-Arena approach [Zheng et al. 2023]. Benchmarks: GSM8K [Cobbe et al. 2021] and MATH [Hendrycks et al. 2021].

Reproducibility

Code and data generators for all three arms are in the repo. Control (`control/gridworld.py`): the MDP, exact value iteration, the noisy-evaluator × lookahead sweep — pure NumPy, exact oracle. Games (`games/`): Connect-4 bitboard engine + solver, the depth-limited alpha-beta oracle, the conv net, MCTS, the GELO calibrator (`c4_calibrate.py` , which prints the goodness-of-fit gate), and the self-play / oracle-value loops. Reasoning (`reasoning/`): the accuracy-vs-N sweep, the evaluator-quality ablation and gradient, the

MATH IRT fit, the pairwise arena + master-judge (Bedrock), the judge-scaling test, and the size×search frontier. Large datasets and model weights are regenerable and not committed.

Appendix A — Experimental setup, per result

So the "what we did" is explicit and reproducible, the exact knobs behind each headline number:

Arm A — Connect-4 / GELO (`games/`). - *Opponent ladder & calibration* (`c4_calibrate.py` , `connect4_ab.py`): opponents are random plus depth-*d* alpha-beta (d = 1…6) against the perfect solver's move ordering. Ratings are fit by logistic MLE from the full round-robin **cross-table** (not per-opponent win-rates); the goodness-of-fit gate is mean |predicted – observed| pairwise win-rate = **0.058**; anchor **random := 0**. - *Evaluator net* (`c4_net.py`): a small conv policy+value network trained on **oracle-solver labels** (value = game-theoretic value, policy = solver move distribution). Data-size sweep uses 12k / 24k / 50k label subsets. - *Placing an agent* (`place_agent`): the agent plays **24–40 games per rung** vs each ladder rung; the win-rate vector is fit to the ladder's GELO by the same logistic model. Open-loop = greedy on the policy head; closed-loop = **PUCT MCTS, 200 simulations**. - *Which-lever diagnostic* (`c4_diagnostic.py`): a data (3k–50k) × capacity (small ≈0.3M / large ≈4M params) grid, each cell placed both open- and closed-loop, to read where data / capacity / search each bind.

Arm B — reasoning (`reasoning/`), all Qwen2.5-Instruct-4bit under MLX. - *Self-consistency & oracle-best-of-N* (`reason_math_sweep.py`): **Qwen-4B**, GSM8K test, **120 problems**, temperature **0.8**, **1024**-token budget, **N ∈ {1,4,16,32}**; final answer via `#### <number>` regex; oracle-best-of-N = "is any of the N correct vs gold." - *Graded verifier* (same samples): a synthetic selector with per-item accuracy **q ∈ {0.5…1.0}**. - *Kimi-best-of-N* (`reason_bestofn.py`): **Qwen-1.5B**, **60 problems**, temp 0.8, **N ∈ {2,4,8}**; selector = **Kimi-K2.5** (Bedrock, temp 0) picking the correct candidate index — blinded to the gold. - *Judge scaling* (`reason_arena.py` judge, majority over repeats): Kimi judgments aggregated over **N ∈ {1…5}**; agreement measured against the ground-truth verifier on decisive pairs. - *Serial axis* (`reason_serial.py`): **greedy** (temp 0), max-tokens ∈ **{128,256,512,1024,2048}**, Qwen-1.5B and 3B. - *Size × search frontier*: Qwen **{0.5B,1.5B,3B}**, GSM8K, greedy + sc@{4,16}; compute proxy ≈ params × N. - *Pairwise reasoning-GELO* (`reason_arena.py`): **5 models** on **50 MATH problems**; **every** ordered pair scored by the Kimi-K2.5 judge (**blinded, answer-order randomized**); ratings by Bradley–Terry MLE (`bt_elo`), anchor **Kimi := 2800**; the verifier cross-check gives the 72% judge-agreement figure. A powered rerun (`reason_arena.py` , resilient concurrent Bedrock judging: shared client with timeouts/retries, thread pool) over 150 MATH problems × the 5-model Qwen2.5 ladder gives **84%** agreement. - *Frontier at scale + MATH crossover* (`reason_cache.py` , `run_cache_ladder.sh`): sample-once-select-many caches — 32 completions/problem via `batch_generate` , then sc@{1,4,16,32}, oracle-best-of-N, and the

graded-verifier curve computed post-hoc. GSM8K ladder 0.5B→72B at n=500; MATH ladder (`--math` , boxed extraction) 0.5B→14B at n=300. - *Difficulty-anchored GELO* (`reason_gelo_irt.py`): the same models vs MATH difficulty tiers L1–L5, Rasch (1-parameter IRT) fit. - *Training-data lever* (`reason_finetune_sweep.py`): LoRA fine-tune Qwen2.5-0.5B-Instruct (8 layers, lr 5e-5) on **N ∈ {64,256,1024,4096}** GSM8K examples at **fixed 3 epochs** (so iters scale with N — isolates *data*, not optimization budget), calculator-annotation targets stripped; greedy accuracy on 150 held-out problems, N=0 = base model.

Arm C — control (`control/gridworld.py`). 8×8 gridworld, slip 0.1, γ = 0.95, scattered pits; exact **V*** by value iteration; degraded evaluator = V* + σ·(value-spread)·$\mathcal{N}(0,1)$; MPC lookahead **h ∈ {1,2,3}** (h Bellman backups, then greedy). Return = mean discounted reward over **400 episodes**, normalized to [random = 0, optimal = 100].

§6 self-improvement. Connect-4 (`c4_selfplay.py`): self-play games each iteration; value target = game **outcome** (baseline) vs **depth-6 oracle value** (`--oracle-value` , the positive control), loop otherwise identical; strength re-placed vs the ladder every iteration. Chess uses the analogous loop from three seeds (from-scratch / weak-policy warm-start / a self-learned +1214 net).

Infrastructure. Two **Apple M3 Ultra** machines (each 275 GB unified memory, ~819 GB/s): **MLX / mlx-lm** for all Qwen inference and net training — batched sampling (`batch_generate`) makes 32-completion caches and models up to **72B** tractable; **AWS Bedrock** (`moonshotai.kimi-k2.5` , us-east-1) for the master judge and the frontier-LLM chess vignette. Early exploratory sweeps used modest samples (50–120 problems); the powered reruns (§3–§5) use 300–500-problem, 32-sample caches and multi-seed/multi-grid averaging, and we flag any result still reported at exploratory power.

References

- Anthony, T., Tian, Z. & Barber, D. (2017). *Thinking Fast and Slow with Deep Learning and Tree Search*. NeurIPS.
- Brown, B. et al. (2024). *Large Language Monkeys: Scaling Inference Compute with Repeated Sampling*. arXiv:2407.21787.
- Browne, C. et al. (2012). *A Survey of Monte Carlo Tree Search Methods*. IEEE TCIAIG.
- Cobbe, K. et al. (2021). *Training Verifiers to Solve Math Word Problems* (GSM8K). arXiv:2110.14168.
- Coulom, R. (2006). *Efficient Selectivity and Backup Operators in Monte-Carlo Tree Search*. Computers and Games.
- DeepSeek-AI (2025). *DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via RL*. arXiv:2501.12948.
- Gulcehre, C. et al. (2023). *Reinforced Self-Training (ReST) for Language Modeling*. arXiv:2308.08998.

- Hendrycks, D. et al. (2021). *Measuring Mathematical Problem Solving with the MATH Dataset*. NeurIPS.
- Jones, A. L. (2021). *Scaling Scaling Laws with Board Games*. arXiv:2104.03113.
- Kocsis, L. & Szepesvári, C. (2006). *Bandit Based Monte-Carlo Planning* (UCT). ECML.
- Lightman, H. et al. (2023). *Let's Verify Step by Step*. arXiv:2305.20050 (ICLR 2024).
- OpenAI (2024). *Learning to Reason with LLMs* (o1).
- Rosin, C. D. (2011). *Multi-Armed Bandits with Episode Context* (PUCT). Ann. Math. AI.
- Silver, D. et al. (2016; 2017; 2018). *Mastering Go / Go without Human Knowledge / a General RL Algorithm* (AlphaGo, AlphaGo Zero, AlphaZero). Nature; Nature; Science.
- Snell, C., Lee, J., Xu, K. & Kumar, A. (2024). *Scaling LLM Test-Time Compute Optimally can be More Effective than Scaling Model Parameters*. arXiv:2408.03314.
- Wang, X. et al. (2023). *Self-Consistency Improves Chain-of-Thought Reasoning*. ICLR.
- Wu, D. J. (2019). *Accelerating Self-Play Learning in Go* (KataGo). arXiv:1902.10565.
- Yao, S. et al. (2023). *Tree of Thoughts*. NeurIPS.
- Yuan, W. et al. (2024). *Self-Rewarding Language Models*. arXiv:2401.10020.
- Zelikman, Y. et al. (2022). *STaR: Bootstrapping Reasoning with Reasoning*. NeurIPS.
- Zheng, L. et al. (2023). *Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena*. NeurIPS.
- **Software/data:** Stockfish · MLX · mlx-lm · AWS Bedrock (Kimi-K2.5) · Lichess cloud evals · HuggingFaceH4/MATH-500.